

BÁO CÁO PHÂN TÍCH MÃ NGUỒN

# MeshGateway

Gateway UART – Wi-Fi – WebSocket của hệ thống MSMS

Nền tảng: ESP32 / ESP-IDF / FreeRTOS

Giao tiếp: UART, Wi-Fi, HTTP, mDNS, WebSocket

Giao diện: LVGL, captive portal

Phiên bản tài liệu: 1.0

Tài liệu được xây dựng từ trạng thái mã nguồn hiện tại

Tháng 07 năm 2026

# Mục lục

|           |                                                          |           |
|-----------|----------------------------------------------------------|-----------|
| <b>1</b>  | <b>Mục đích và phạm vi</b>                               | <b>1</b>  |
| <b>2</b>  | <b>Bối cảnh và ranh giới hệ thống</b>                    | <b>1</b>  |
| 2.1       | Sơ đồ nguồn và đích dữ liệu . . . . .                    | 1         |
| 2.2       | Sơ đồ khối phần cứng và giao tiếp . . . . .              | 2         |
| <b>3</b>  | <b>Tổ chức mã nguồn</b>                                  | <b>2</b>  |
| <b>4</b>  | <b>Kiến trúc phần mềm nội bộ</b>                         | <b>3</b>  |
| 4.1       | Quan hệ giữa các task và trạng thái dùng chung . . . . . | 3         |
| 4.2       | Trình tự khởi động . . . . .                             | 4         |
| <b>5</b>  | <b>Luồng UART giữa root và gateway</b>                   | <b>4</b>  |
| 5.1       | Cấu hình và giao thức bắt tay . . . . .                  | 4         |
| 5.2       | Đường ống xử lý dữ liệu UART . . . . .                   | 4         |
| <b>6</b>  | <b>Wi-Fi, captive portal và mDNS</b>                     | <b>6</b>  |
| 6.1       | Máy trạng thái kết nối Wi-Fi . . . . .                   | 6         |
| <b>7</b>  | <b>WebSocket local và server</b>                         | <b>7</b>  |
| 7.1       | Quy tắc chọn giao thức . . . . .                         | 7         |
| 7.2       | Vòng đời WebSocket . . . . .                             | 7         |
| 7.3       | Trình tự gửi dữ liệu đầu cuối . . . . .                  | 8         |
| <b>8</b>  | <b>Nguồn tạo bản tin gateway_status</b>                  | <b>8</b>  |
| <b>9</b>  | <b>Màn hình và dữ liệu cục bộ</b>                        | <b>9</b>  |
| <b>10</b> | <b>Phân vùng flash và OTA</b>                            | <b>11</b> |
| <b>11</b> | <b>Tài nguyên FreeRTOS</b>                               | <b>11</b> |
| <b>12</b> | <b>Rủi ro và đề xuất bảo trì</b>                         | <b>11</b> |
| <b>13</b> | <b>Quy trình build và kiểm thử</b>                       | <b>12</b> |

|                                |           |
|--------------------------------|-----------|
| <b>14 Kết luận</b>             | <b>13</b> |
| <b>A Thứ tự đọc mã đề xuất</b> | <b>13</b> |

## Danh sách hình vẽ

|    |                                                                               |    |
|----|-------------------------------------------------------------------------------|----|
| 1  | Bối cảnh và đường đi đầu cuối của dữ liệu . . . . .                           | 1  |
| 2  | Sơ đồ khối phần cứng quanh gateway . . . . .                                  | 2  |
| 3  | Task, queue và vùng trạng thái dùng chung . . . . .                           | 3  |
| 5  | Máy trạng thái bắt tay UART . . . . .                                         | 4  |
| 4  | Lưu đồ khởi động thực tế . . . . .                                            | 5  |
| 7  | Luồng STA/AP và captive portal . . . . .                                      | 6  |
| 8  | Phân nhánh local WS và server WSS . . . . .                                   | 7  |
| 9  | Máy trạng thái vòng đời WebSocket . . . . .                                   | 7  |
| 11 | Nguồn dữ liệu tạo trạng thái gateway . . . . .                                | 8  |
| 12 | Một snapshot metrics phục vụ cả màn hình và WebSocket . . . . .               | 9  |
| 6  | Đường ống từ byte UART tới bản tin WebSocket . . . . .                        | 10 |
| 10 | Sequence diagram từ root đến dashboard . . . . .                              | 10 |
| 13 | Các partition flash và kích thước khai báo . . . . .                          | 11 |
| 14 | Trạng thái triển khai OTA: partition có sẵn nhưng pipeline chưa nổi . . . . . | 11 |

## Danh sách bảng

|   |                                                 |    |
|---|-------------------------------------------------|----|
| 1 | Các module quan trọng của MeshGateWay . . . . . | 2  |
| 2 | Các trường của gateway_status . . . . .         | 8  |
| 3 | Các task chính nhìn thấy trong mã . . . . .     | 12 |

# 1 Mục đích và phạm vi

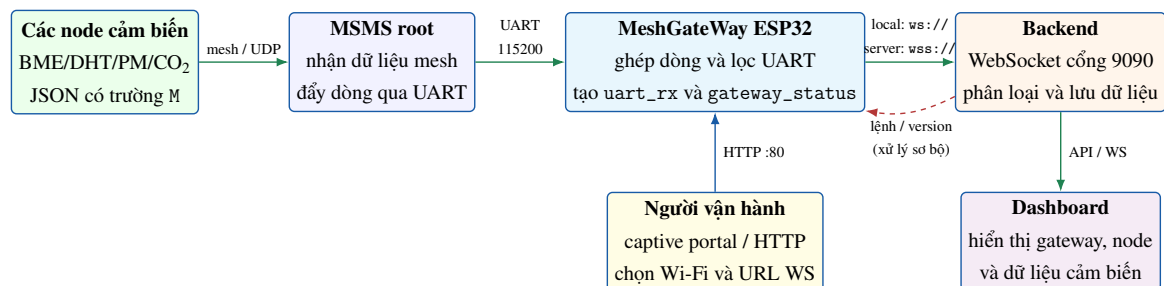
MeshGateWay là firmware trung gian giữa nút root của mạng cảm biến và hệ thống máy chủ. Gateway không trực tiếp vận hành mạng mesh trong luồng chính hiện tại. Nó nhận chuỗi telemetry từ root qua UART, bổ sung trạng thái của chính gateway, rồi chuyển dữ liệu lên backend bằng WebSocket.

Tài liệu tập trung vào mã đang được gọi từ `app_main()`. Những module có trong cây mã nhưng chưa được nối vào luồng chạy, chẳng hạn FOTA hoàn chỉnh, ESP\_Bus và một số tiện ích cũ, được ghi rõ là hạ tầng dự phòng thay vì mô tả như chức năng đã hoạt động.

## 2 Bối cảnh và ranh giới hệ thống

### 2.1 Sơ đồ nguồn và đích dữ liệu

Hình 1 trả lời ba câu hỏi: dữ liệu sinh ra ở đâu, gateway xử lý gì, và dữ liệu được tiêu thụ ở đâu. Mũi tên xanh lá là luồng telemetry; mũi tên xanh dương là điều khiển hoặc cấu hình.

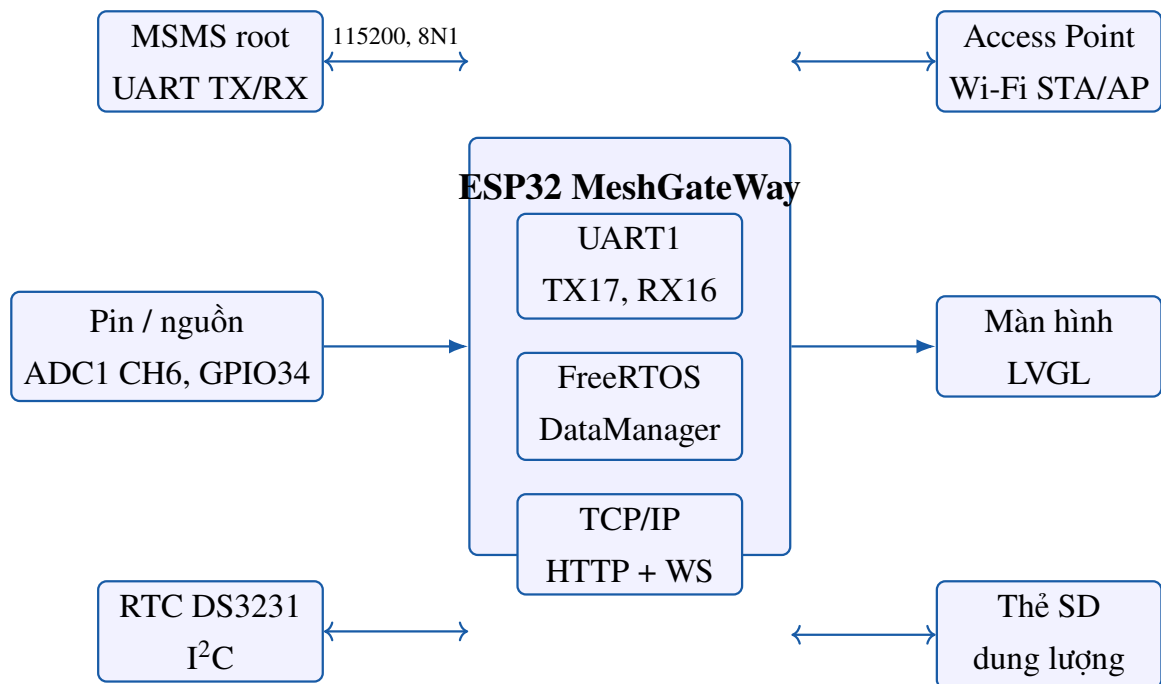


Hình 1: Bối cảnh và đường đi đầu cuối của dữ liệu

Gateway có hai nguồn dữ liệu độc lập:

- **Nguồn từ node:** chuỗi JSON đi qua UART, được giữ nguyên trong trường payload của bản tin `uart_rx`.
- **Nguồn nội tại gateway:** CPU, RAM, pin, uptime, SSID, RSSI, IP và gateway mạng được đo tại gateway và gửi bằng `gateway_status`.

## 2.2 Sơ đồ khối phần cứng và giao tiếp



Hình 2: Sơ đồ khối phần cứng quanh gateway

## 3 Tổ chức mã nguồn

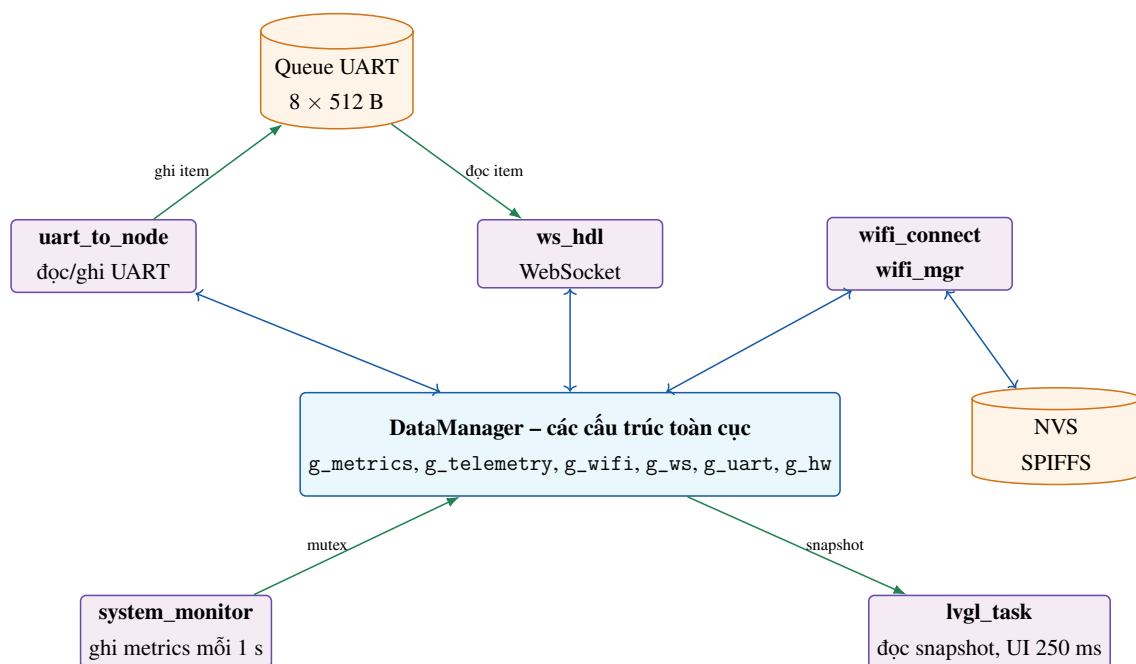
Bảng 1: Các module quan trọng của MeshGateWay

| Đường dẫn                           | Trách nhiệm                                                                              |
|-------------------------------------|------------------------------------------------------------------------------------------|
| main/MeshGateWay.c                  | Điểm vào, khởi tạo NVS, SPIFFS, phần cứng và các task.                                   |
| components/core/<br>Datamanager     | Các cấu trúc trạng thái dùng chung cho phần cứng, metrics, Wi-Fi, WS, UART và telemetry. |
| components/core/<br>UartToNode      | Bắt tay root, đọc UART, lọc và đẩy dữ liệu vào queue.                                    |
| components/core/<br>WebSocketHandle | Chọn URL, phân giải mDNS, duy trì client WS/WSS, gửi dữ liệu.                            |
| components/core/<br>WifiManager     | STA/AP, captive portal, DNS, HTTP và mDNS meshgateway.local.                             |

| Đường dẫn                         | Trách nhiệm                                           |
|-----------------------------------|-------------------------------------------------------|
| components/core/<br>SystemMonitor | Thu thập metrics mỗi giây.                            |
| components/core/<br>ScreenManager | LVGL, hiển thị trạng thái và biểu đồ RX/TX.           |
| components/core/<br>PowerManager  | Đọc ADC và ước lượng điện áp/phần trăm pin.           |
| components/drivers/SD_<br>Card    | Khởi tạo và đọc dung lượng thẻ SD.                    |
| components/core/<br>FOTAManager   | Mã OTA cũ/chưa được nối vào luồng chạy hiện tại.      |
| components/core/<br>WebConfigWifi | Tệp HTML được đóng gói vào SPIFFS cho captive portal. |

## 4 Kiến trúc phần mềm nội bộ

### 4.1 Quan hệ giữa các task và trạng thái dùng chung



Hình 3: Task, queue và vùng trạng thái dùng chung



`g_metrics` được bảo vệ bằng mutex. Các biến đếm telemetry được nhiều task truy cập trực tiếp; nếu sau này tăng mức đồng thời hoặc yêu cầu số liệu tuyệt đối, nên chuyển các bộ đếm sang thao tác atomic hay mutex riêng.

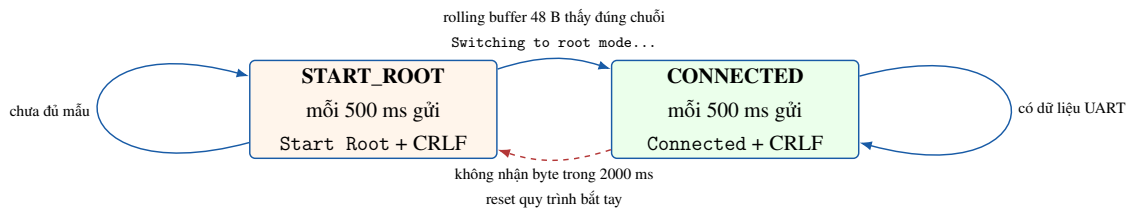
## 4.2 Trình tự khởi động

Điểm đáng chú ý là `wifi_init_sta()` thực hiện thử kết nối đồng bộ trước khi trả về. Vì vậy thời gian boot có thể tăng đáng kể khi nhiều SSID đều không khả dụng. SNTP cũng chờ trong `app_main()`, nhưng các task monitor, Wi-Fi, WebSocket và UART đã được tạo trước đó nên vẫn có thể chạy.

# 5 Luồng UART giữa root và gateway

## 5.1 Cấu hình và giao thức bắt tay

UART mặc định dùng UART1, 115200 baud, 8N1, TX GPIO17, RX GPIO16, không dùng RTS/CTS. Driver có RX buffer 2048 B, TX buffer 1024 B và event queue 64 phần tử.



Hình 5: Máy trạng thái bắt tay UART

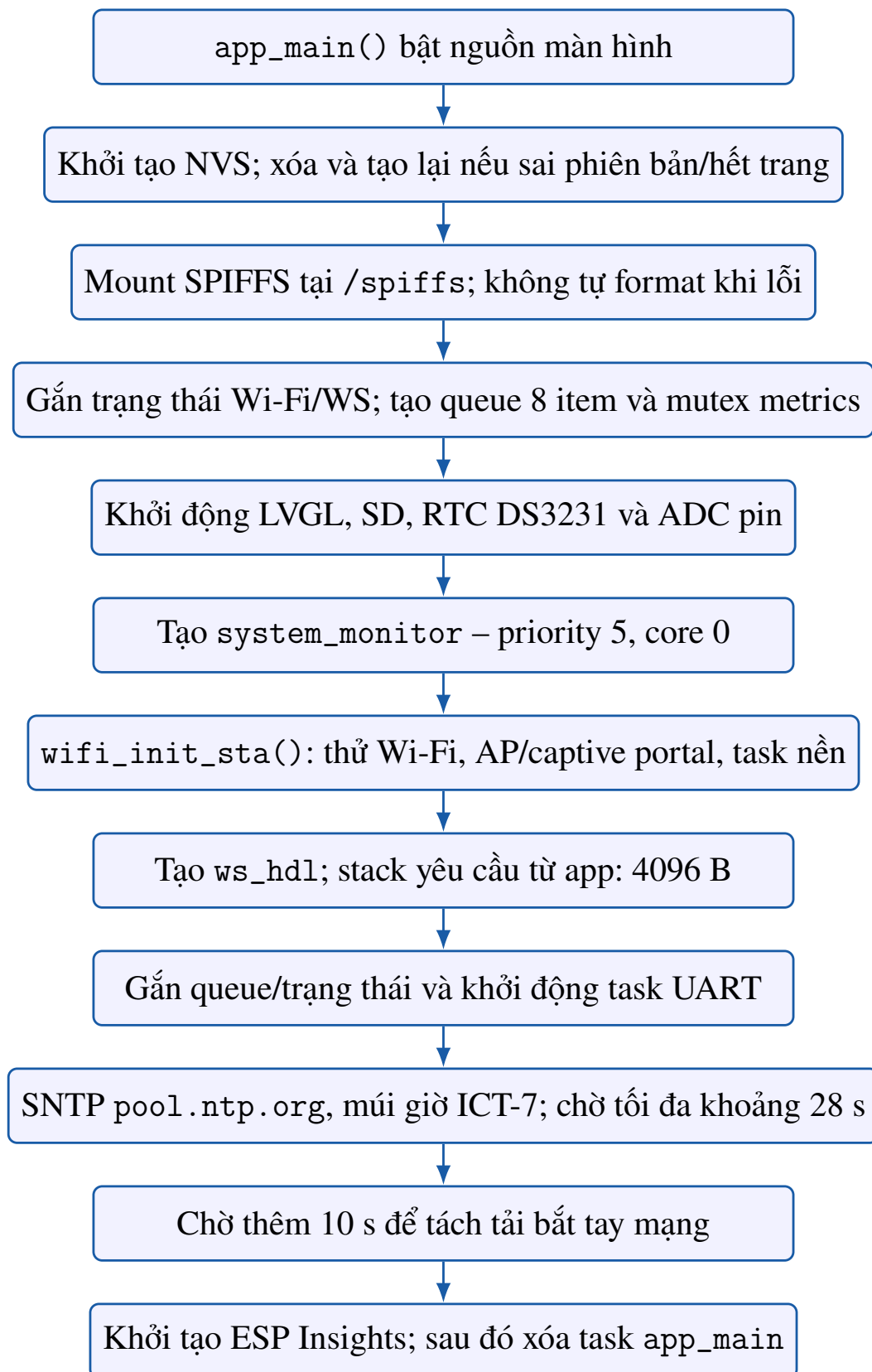
Chuỗi xác nhận phải khớp chính xác `Switching to root mode...` và kết thúc bằng CRLF. Rolling buffer cho phép nhận chuỗi qua nhiều event UART. Sau khi kết nối, nếu đường UART im lặng quá 2 giây, gateway quay lại `START_ROOT`.

## 5.2 Đường ống xử lý dữ liệu UART

Queue có sức chứa tải danh nghĩa tối đa  $8 \times 512 = 4096$  byte, chưa tính metadata. Queue này chỉ giảm chênh lệch tốc độ ngắn hạn; nó không phải kho dữ liệu offline. Nếu WebSocket chưa kết nối, hoặc queue đầy, telemetry có thể mất.

Listing 1: Dạng bản tin uplink UART

```
{
```



Hình 4: Lưu đồ khởi động thực tế

```

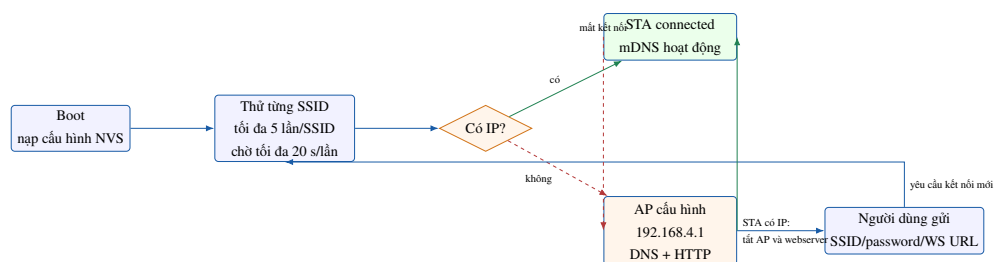
"type": "uart_rx",
"len": 127,
"payload": "{\v\":1,\"seq\":88,\"n\":2,\"M\":\B4:BF:E9:34:0C:CC
  \",...}"
}

```

Trường M trong payload là định danh MAC của node gửi. Gateway không đổi M; backend cần parse lớp JSON bên ngoài, sau đó parse tiếp chuỗi payload để phân tách node và các điểm đo.

## 6 Wi-Fi, captive portal và mDNS

### 6.1 Máy trạng thái kết nối Wi-Fi



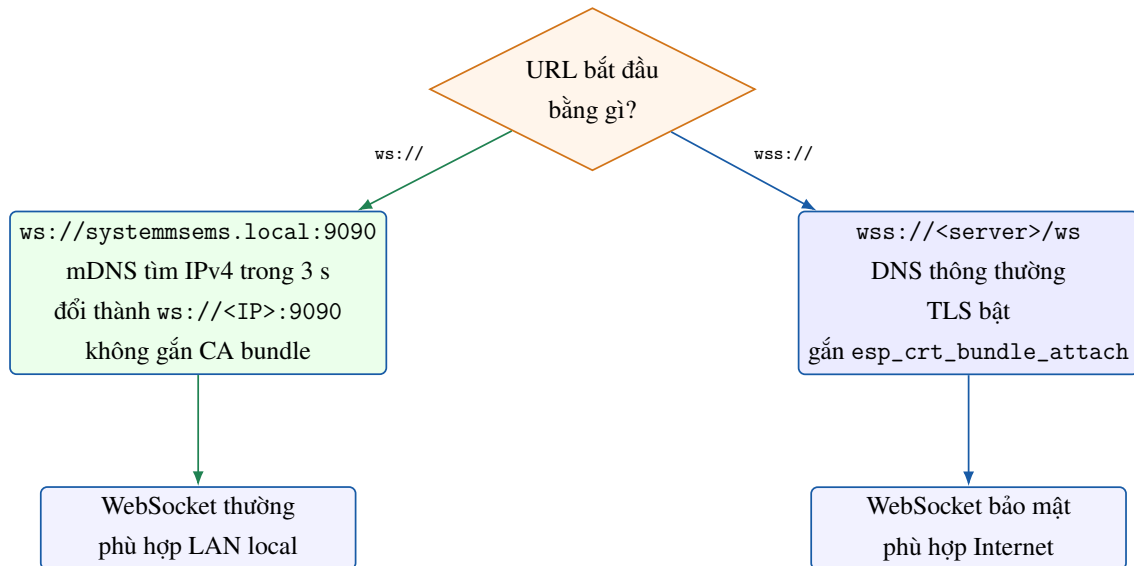
Hình 7: Luồng STA/AP và captive portal

Khi STA lấy được IP, firmware khởi tạo mDNS với hostname `meshgateway.local` và công bố dịch vụ HTTP cổng 80. Đây là tên của **gateway và trang cấu hình**, khác với `systemmsems.local` là tên máy chạy backend local.

Danh sách Wi-Fi boot hiện chứa cả credential được viết trực tiếp trong `wifi_manager_core.c`. Đây là rủi ro bảo mật; credential nên chuyển sang NVS provisioning hoặc cấu hình bí mật ngoài repository.

## 7 WebSocket local và server

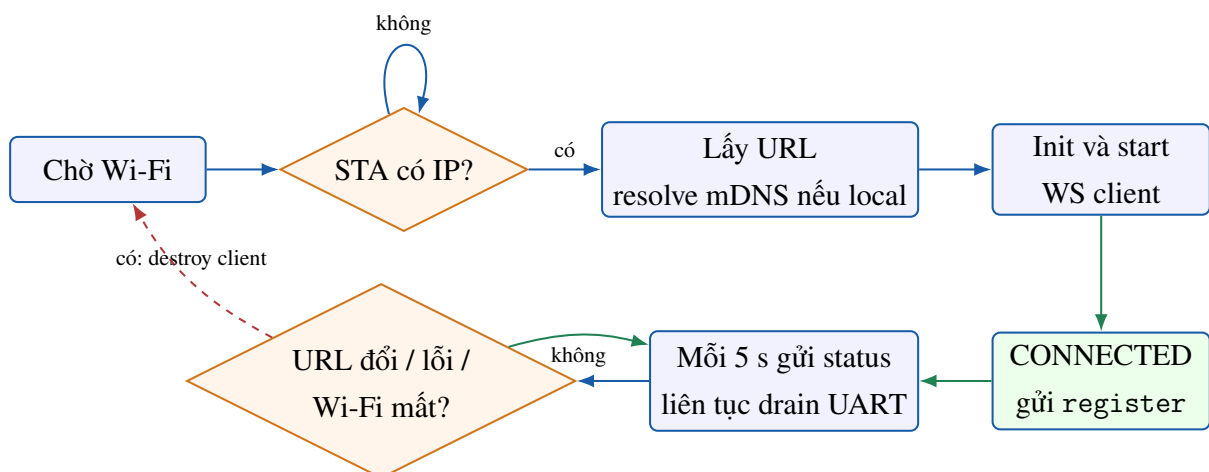
### 7.1 Quy tắc chọn giao thức



Hình 8: Phân nhánh local WS và server WSS

Frontend chạy tại localhost:3000 không buộc ESP32 dùng WSS. Giao thức của ESP32 phụ thuộc URL backend được cấu hình: local hiện dùng `ws://systemmsems.local:9090`; server Internet dùng `wss://`. HTTP của frontend và WebSocket của gateway là hai kết nối độc lập.

### 7.2 Vòng đời WebSocket

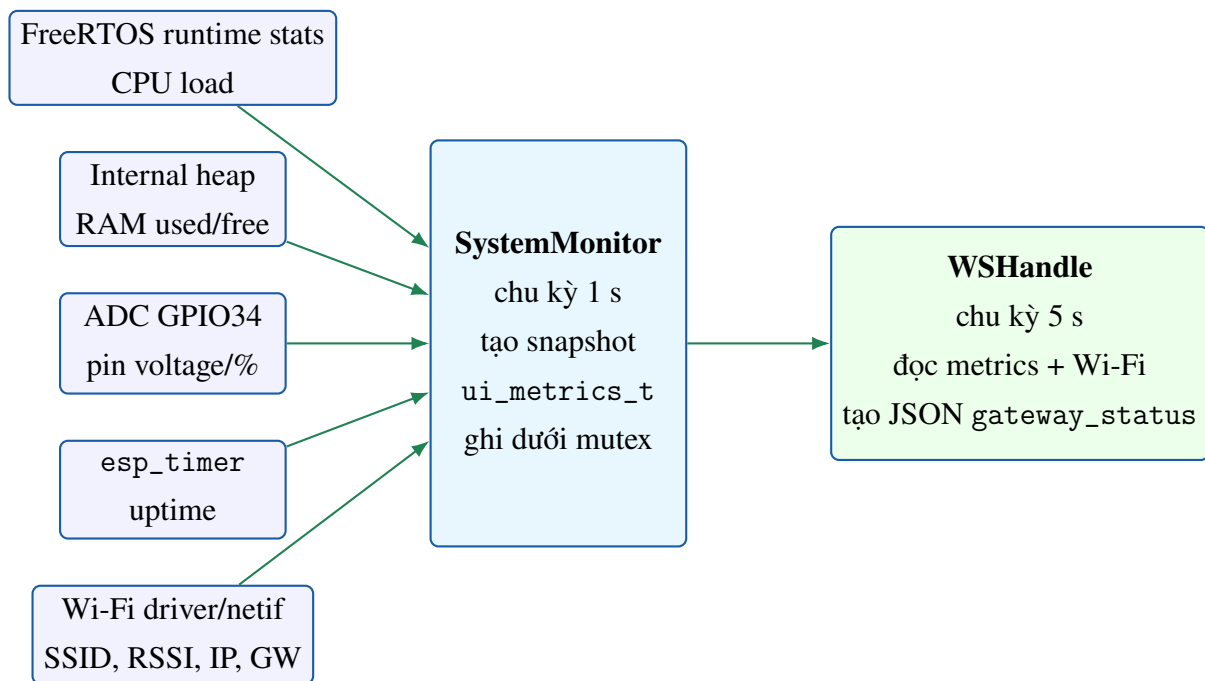


Hình 9: Máy trạng thái vòng đời WebSocket

Client dùng timeout mạng và reconnect 15 giây, buffer 2048 B và keepalive. `save_ws_url()` cập nhật URL và yêu cầu restart client, nhưng chỉ ghi vào RAM. Sau reset, URL quay lại lựa chọn trong `Kconfig/sdkconfig`. Vì vậy thành cấu hình frontend hiện thay đổi được nguồn trong phiên chạy, chưa phải cấu hình bền vững.

### 7.3 Trình tự gửi dữ liệu đầu cuối

## 8 Nguồn tạo bản tin `gateway_status`



Hình 11: Nguồn dữ liệu tạo trạng thái gateway

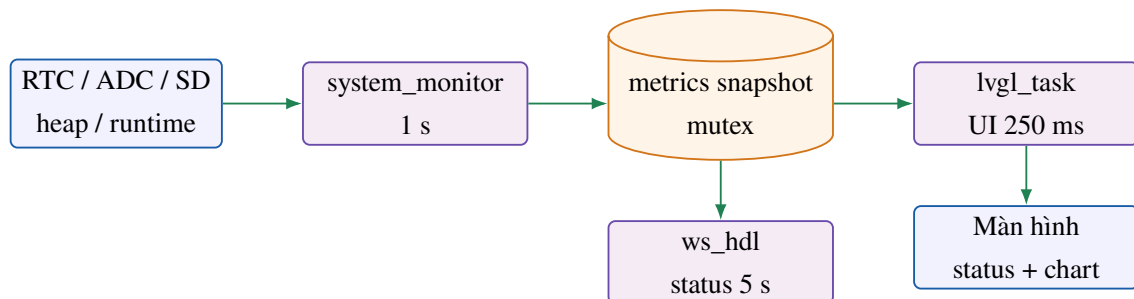
Bảng 2: Các trường của `gateway_status`

| Trường                                                           | Nguồn              | Ý nghĩa                               |
|------------------------------------------------------------------|--------------------|---------------------------------------|
| <code>cpu_load_percent</code>                                    | Runtime stats      | Tải CPU trung bình, đơn vị %.         |
| <code>ram_used_kb</code> ,<br><code>ram_used_percent</code>      | Internal heap      | RAM nội bộ đã dùng.                   |
| <code>battery_voltage_v</code> ,<br><code>battery_percent</code> | ADC + PowerManager | Điện áp gói pin và mức pin ước lượng. |

| Trường                  | Nguồn             | Ý nghĩa                                 |
|-------------------------|-------------------|-----------------------------------------|
| power_source            | Điện áp pin       | Heuristic: unknown, battery, AC_or_USB. |
| chip_temp_c             | Cảm biến nội chip | Có thể là null nếu SoC không hỗ trợ.    |
| uptime_s                | esp_timer         | Số giây từ lần boot/reset.              |
| wifi_ssid,<br>wifi_rssi | Wi-Fi AP record   | Mạng đang kết nối và RSSI dBm.          |
| sta_ip, sta_gateway     | STA netif         | IPv4 của gateway và router.             |

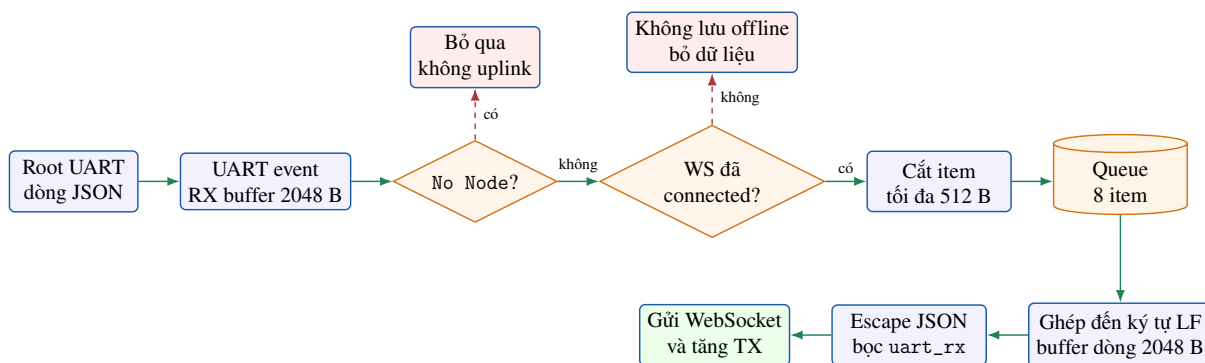
## 9 Màn hình và dữ liệu cục bộ

LVGL đọc snapshot metrics và cập nhật UI khoảng mỗi 250 ms. Các giá trị RX/TX, IP, MAC rút gọn và mức sử dụng queue được cập nhật theo chu kỳ 1 giây. SystemMonitor đo RTC, pin, SD, heap, CPU, FPS, uptime và nhiệt độ chip mỗi giây.

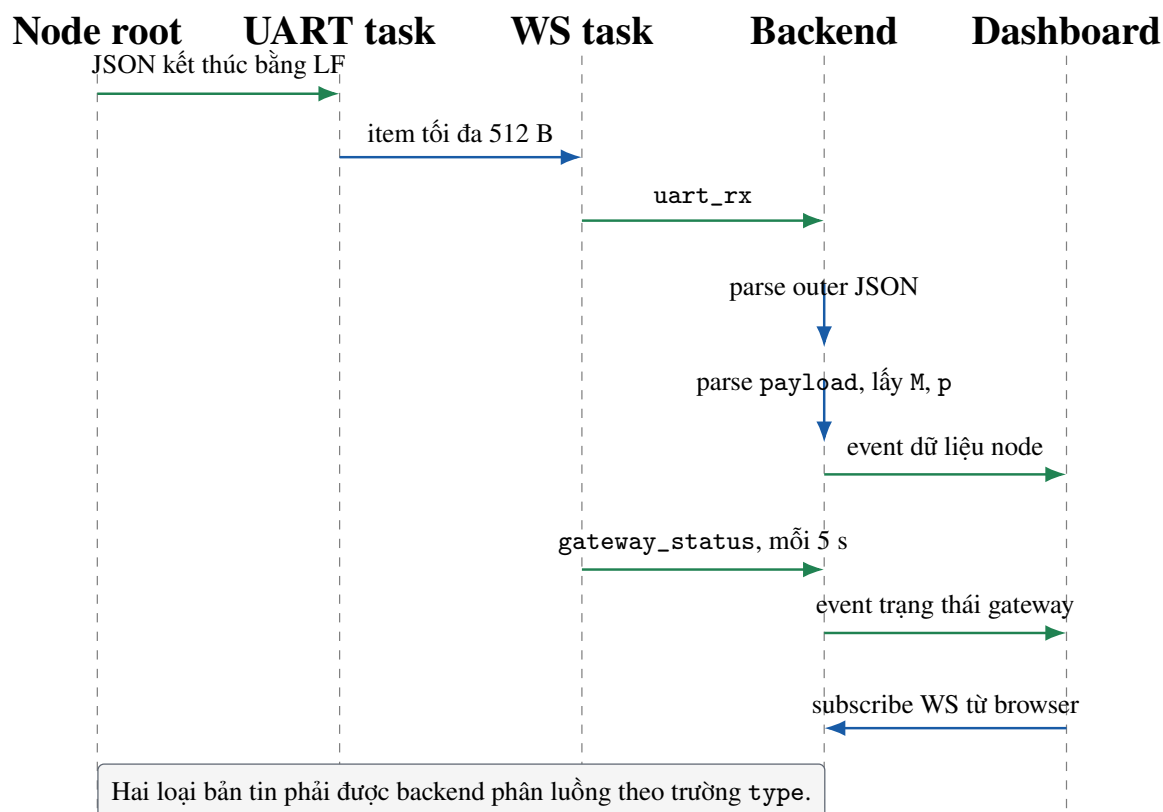


Hình 12: Một snapshot metrics phục vụ cả màn hình và WebSocket

SPIFFS hiện lưu tài nguyên captive portal. Thẻ SD được khởi tạo và đo dung lượng, nhưng đường nhận UART–WebSocket hiện không ghi telemetry vào SD, SQLite hay MongoDB. Việc lưu dữ liệu dài hạn thuộc backend; gateway chỉ chuyển tiếp trong RAM.



Hình 6: Đường ống từ byte UART tới bản tin WebSocket



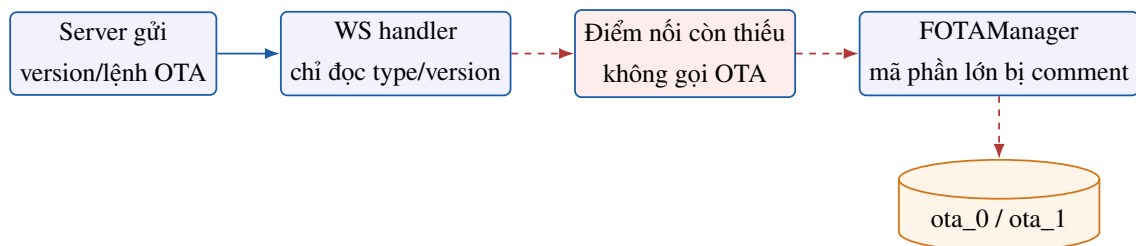
Hình 10: Sequence diagram từ root đến dashboard

## 10 Phân vùng flash và OTA

|                      |                          |                          |                          |                          |
|----------------------|--------------------------|--------------------------|--------------------------|--------------------------|
| <b>NVS</b><br>0x5000 | <b>otadata</b><br>0x2000 | <b>ota_0</b><br>0x1C0000 | <b>ota_1</b><br>0x1C0000 | <b>SPIFFS</b><br>0x20000 |
|----------------------|--------------------------|--------------------------|--------------------------|--------------------------|

Sơ đồ ưu tiên khả năng đọc; chiều rộng NVS, otadata và SPIFFS đã được phóng đại.

Hình 13: Các partition flash và kích thước khai báo



Hình 14: Trạng thái triển khai OTA: partition có sẵn nhưng pipeline chưa nổi

Không nên hiểu sự tồn tại của hai partition OTA là FOTA đã hoạt động. Trong luồng hiện tại, WebSocket chỉ lưu/log type và version; không gọi hàm tải firmware. Khi hoàn thiện cần thêm xác thực manifest, kiểm tra phiên bản, chữ ký/hash, chính sách certificate và rollback.

## 11 Tài nguyên FreeRTOS

SystemMonitor cấp phát mảng TaskStatus\_t bằng malloc() mỗi giây để tính CPU load. Có thể giảm phân mảnh heap bằng buffer tái sử dụng hoặc chu kỳ đo dài hơn.

## 12 Rủi ro và đề xuất bảo trì

- Bí mật trong mã nguồn:** auth key ESP Insights và một số mật khẩu Wi-Fi đang viết trực tiếp trong tệp C. Cần thu hồi/đổi khóa đã lộ và chuyển sang provisioning, biến môi trường build hoặc kho bí mật.
- URL WebSocket không bền vững:** thay đổi qua web chỉ lưu RAM. Nên lưu URL hợp lệ vào NVS và tải trước khi tạo client.
- Không có lưu trữ offline:** mất WS đồng nghĩa bỏ telemetry UART. Có thể bổ sung ring buffer trong RAM hoặc journal trên SD, kèm giới hạn dung lượng.



Bảng 3: Các task chính nhìn thấy trong mã

| Task           | Stack              | Priority | Vai trò                                                                                   |
|----------------|--------------------|----------|-------------------------------------------------------------------------------------------|
| system_monitor | 3072 B             | 5        | Metrics, pin, RTC, SD, heap, CPU; pin core 0.                                             |
| lvgl_task      | 4096 B             | 4        | Render và cập nhật màn hình; pin core 1.                                                  |
| ws_hdl         | 4096 B             | 5        | Quản lý client và gửi dữ liệu. Client WS nội bộ yêu cầu stack 6144 cho task của thư viện. |
| uart_to_node   | 4096 B<br>mặc định | 5        | Bắt tay và đọc/ghi UART.                                                                  |
| wifi_connect   | 4096 B             | 5        | Áp dụng credential mới.                                                                   |
| wifi_mgr       | 4096 B             | 5        | Giám sát STA/AP và reconnect.                                                             |
| dns_server     | 2560 B             | 5        | DNS captive portal khi AP hoạt động.                                                      |

4. **Tách JSON hai lớp:** backend phải parse `uart_rx.payload`. Nên định nghĩa schema và test contract cho trường `M`, `p`, `seq` và `err`.
5. **Danh sách Wi-Fi boot làm chậm khởi động:** nhiều lần chờ 20 giây có thể kéo dài boot. Nên scan trước, chỉ thử SSID nhìn thấy và giới hạn ngân sách thời gian.
6. **FOTA chưa hoàn chỉnh:** không mở chức năng OTA cho sản phẩm trước khi có xác thực, rollback và kiểm thử mất điện.
7. **mDNS phụ thuộc mạng LAN:** `systemmsems.local` chỉ phân giải khi máy backend quảng bá mDNS và router không chặn multicast. Nên cho phép nhập IP/hostname thay thế và hiển thị lỗi phân giải rõ ràng.

## 13 Quy trình build và kiểm thử

Listing 2: Build và flash firmware

```
cd MeshGateWay
idf.py set-target esp32
idf.py menuconfig
idf.py build
idf.py -p COMx flash monitor
```

Các ca kiểm thử tối thiểu:

- boot khi có/không có Wi-Fi; kiểm tra AP, DNS captive portal và `meshgateway.local`;
- `local ws://systemmsems.local:9090` và `server wss://...`; xác minh log hiển

thì plain hoặc TLS;

- ngắt UART quá 2 giây để kiểm tra quay lại START\_ROOT;
- gửi JSON dài, chia qua nhiều chunk và xác minh backend nhận đúng một dòng;
- làm đầy queue hoặc ngắt WS để đo số bản tin bị mất;
- đối chiếu gateway\_status với RSSI, IP, pin và heap thực tế;
- reset sau khi đổi URL để xác nhận giới hạn lưu RAM hiện tại.

## 14 Kết luận

MeshGateWay được tổ chức thành ba đường xử lý chính: UART nhận dữ liệu từ root, SystemMonitor tạo metrics nội tại, và WebSocket hợp nhất hai nguồn để gửi backend. FreeRTOS queue tách task UART khỏi task mạng; mutex tạo snapshot ổn định cho UI và status. Local dùng ws:// qua mDNS, còn server dùng wss:// với TLS. Các giới hạn quan trọng nhất của phiên bản hiện tại là không lưu telemetry offline, URL WebSocket chỉ lưu RAM, FOTA chưa nổi và còn bí mật được viết trực tiếp trong mã nguồn.

## A Thứ tự đọc mã đề xuất

| Tệp                                                      | Nội dung nên đọc                             |
|----------------------------------------------------------|----------------------------------------------|
| main/MeshGateWay.c                                       | Trình tự khởi động và quan hệ tổng thể.      |
| components/core/<br>Datamanager/Datamanager.h            | Toàn bộ kiểu trạng thái và kích thước queue. |
| components/core/<br>UartToNode/UartToNode.c              | Máy trạng thái bắt tay và uplink UART.       |
| components/core/<br>WebSocKetHandle/WSHandle.<br>c       | URL, mDNS, WS/WSS và schema bản tin.         |
| components/core/<br>WifiManager/wifi_manager_<br>core.c  | Event Wi-Fi, danh sách boot và mDNS.         |
| components/core/<br>WifiManager/wifi_manager_<br>tasks.c | Reconnect và chuyển STA/AP.                  |

| Tệp                                                                    | Nội dung nên đọc                       |
|------------------------------------------------------------------------|----------------------------------------|
| <code>components/core/<br/>WifiManager/wifi_manager_<br/>http.c</code> | Endpoint cấu hình web.                 |
| <code>components/core/<br/>SystemMonitor/<br/>SystemMonitor.c</code>   | Cách tạo metrics.                      |
| <code>components/core/<br/>ScreenManager/<br/>ScreenManager.c</code>   | Cách metrics được đưa lên LVGL.        |
| <code>partitions.csv</code>                                            | Phân vùng NVS, hai OTA slot và SPIFFS. |